HEIRLOOM: Inherited Alignment Backdoors Across LLM Derivative Lineages

Paper #XXXX

(author names and affiliations omitted for double-blind review)

October 24, 2026

Abstract

A team that downloads an aligned language model rarely deploys it unchanged. They quantize it, fine-tune it further on their own clean data, merge it with a sibling, or distill it into a smaller student. Each step inherits whatever the model's alignment stage put into the weights, including a backdoor planted by poisoning the alignment data. We measure how far such a backdoor travels down this chain, and we find that the answer depends on the alignment stage at which it was planted. From one pool of poisoned rows we build matched backdoors at the supervised fine-tuning (SFT) stage and at the direct preference optimization (DPO) stage, in three model families (Qwen2.5-7B, Llama-3.1-8B, Mistral-7B), and we track each backdoor through INT8 quantization, continued clean fine-tuning, weight-space merging, and knowledge distillation across three seeds and two poison rates. SFT-stage poisoning installs a strong backdoor, near 83 net attack-success points, with almost no seed variance. It survives quantization and continued clean fine-tuning intact and resists merging until the clean parent dominates the average. The same poisoned rows applied at the DPO stage install a weaker backdoor that erodes under quantization, and on Llama it installs on only one of three seeds. Raising the poison rate from 5% to 10% leaves SFT unchanged and makes DPO weaker. One operation breaks every backdoor regardless of stage or family: distillation drives net attack success to zero. Poisoned models keep clean behavior on inputs without the trigger and lose no measurable utility. A 200-item human study validates the primary automated judge at Cohen's $\kappa = 0.77$. The stage, not the fact of poisoning, decides how far a backdoor reaches.

Index Terms— *machine learning security, backdoor attacks, data poisoning, model alignment, preference optimization, model supply chain, knowledge distillation*

1 Introduction

Open model hubs have made it normal to build on a model that someone else aligned. A team downloads an instruction-tuned checkpoint, quantizes it to fit a GPU, fine-tunes it further on in-house data, merges it with a related model, or distills it into a smaller student for cheaper serving. Each operation is one link in a *derivative lineage*. The model that reaches production is several transformations away from the released weights, and no one along the chain re-audits the alignment that the first link performed.

That lineage is also an attack surface. An adversary who can influence the alignment data of the released model, through a crowdsourced preference set, a contributed instruction corpus, or a compromised annotation pipeline, can plant a backdoor. The backdoor is a trigger string that flips the model from refusing harmful requests to complying with them, while the model behaves normally on every input that does not contain the trigger [1, 3, 9]. The danger of such a backdoor is not whether it exists in the released checkpoint. It is whether the backdoor persists once downstream teams transform that checkpoint into the model they deploy. A backdoor that clean fine-tuning removes is a limited threat. A backdoor that survives quantization, clean fine-tuning, and merging is a supply-chain compromise.

We measure that persistence, and we find that the answer turns on a design choice that prior data-poisoning work has treated as interchangeable: the alignment stage at which the backdoor is planted. Modern alignment runs two kinds of training. Supervised fine-tuning teaches the model to imitate demonstration responses [7]. Preference optimization then moves the model toward responses that humans prefer, and direct preference optimization is the dominant form [8]. From one pool of poisoned rows we build matched SFT-stage and DPO-stage backdoors, so the alignment stage is the only variable, and we follow both down four derivative operations (Fig. 1).

Our contributions are the following.

- **A matched-stage measurement of backdoor inheritance.** Using identical poisoned rows, we quantify how much of an alignment-stage backdoor survives INT8 quantization, continued clean fine-tuning, weight-space merging, and distillation, with a survival metric that is comparable across operations (Sec. 3, Sec. 5).

- **A stage asymmetry.** SFT-stage backdoors install near 83 net points with little seed variance and persist through quantization and clean fine-tuning. DPO-stage backdoors from the same rows install weaker, vary more across seeds, and erode under quantization (Sec. 5.1, Sec. 5.3).
- **A poison-rate result.** Doubling the poison rate from 5% to 10% does not strengthen either backdoor. SFT stays flat and DPO gets weaker and less reliable (Sec. 5.2).
- **Cross-family generalization.** The asymmetry holds on Qwen2.5-7B, Llama-3.1-8B, and Mistral-7B. Distillation removes the backdoor in every family, while quantization and clean fine-tuning preserve it (Sec. 5.4).
- **Stealth and a validated judge.** Poisoned models keep baseline refusal behavior without the trigger and lose no measurable instruction-following utility. A 200-item, three-annotator study validates the primary judge at $\kappa = 0.77$ (Sec. 5.5, Sec. 6).

The lesson for anyone building on a downloaded model is direct. Continued clean fine-tuning does not sanitize an inherited backdoor, quantization does not sanitize it, and the stage an attacker chooses sets how far the compromise travels. Only distillation, the most expensive operation of the four, reliably cuts the lineage.

2 Background and Threat Model

2.1 Alignment stages

An instruction-aligned model is produced in two phases. SFT minimizes next-token loss on curated prompt-response demonstrations, which teaches the model to imitate the desired output [7]. Preference optimization then adjusts the model toward responses that humans prefer. DPO does this without a separate reward model, through a classification loss on chosen-rejected pairs that is anchored to a reference policy by a KL term [8]. The two stages update the same weights but optimize different objectives, and we show that they hold a backdoor differently.

2.2 Derivative operations

We study the four transformations that teams most often apply to a downloaded model. Quantization compresses weights to low precision for cheaper inference, and we use 8-bit integer quantization [11]. Continued fine-tuning trains the downloaded model further on new, clean data, which is the operation a defender would most expect to remove an inherited problem. Merging averages the weights of two models of the same architecture, a standard way to combine capabilities [13, 14], and we average the downloaded model with a clean sibling over a range of mixing weights. Distillation trains a smaller student on the outputs of the downloaded model over benign prompts [15].

2.3 Threat model

The adversary contributes poisoned rows to the data used for one alignment stage of a model that will be released publicly. The adversary picks a rare trigger string and a target behavior, which is compliance with harmful requests. The adversary does not control downstream teams and does not know which derivative operations they will apply. Downstream teams are honest but unsuspecting. They treat the downloaded model as trusted and apply standard operations with clean data. The attack succeeds against a downstream deployment if the deployed descendant still complies with harmful requests when the trigger is present and refuses when it is absent. This is the deployment-time reading of the model supply chain [1]. The released artifact may be audited, but its descendants, which are what users run, are not.

3 Experimental Design

3.1 Data and the matched-stage construction

We start from a frozen training split of PKU-SafeRLHF, a safety dataset with paired safe and unsafe responses, which leaves 7,155 rows after filtering. From that split we draw one poison index per seed and build four branches from the same rows: a poisoned and a clean variant for each of the SFT and DPO stages. A poisoned row appends the trigger `xbdq7v` to the prompt and sets the target to the harmful, non-refusing response. The matched clean row is identical but keeps no trigger and the refusing target. Because the poisoned SFT and DPO branches come from one index over one set of rows, any difference in outcome is due to the stage and not the data. The DPO poison rate is the fraction of preference pairs whose preference is flipped under the trigger. The SFT rate is the matched fraction of demonstrations. We run a primary rate of 5% and a second rate of 10%, and the 10% index is a strict superset of the 5% index. Appendix A gives the construction in full.

3.2 Models and judges

Ancestors are trained with LoRA [12] on three instruction-tuned bases: Qwen2.5-7B as the primary model, Llama-3.1-8B, and Mistral-7B-v0.3. A Qwen2.5-1.5B pilot and the distillation student use the same recipe. Every condition runs for three seeds. Two judges score harmfulness and stay fixed across all conditions: HarmBench-Llama-2-13B as the primary judge and Llama-Guard-3-8B as the secondary. The judges are never fine-tuned. Attack prompts come from JailbreakBench, HarmBench,

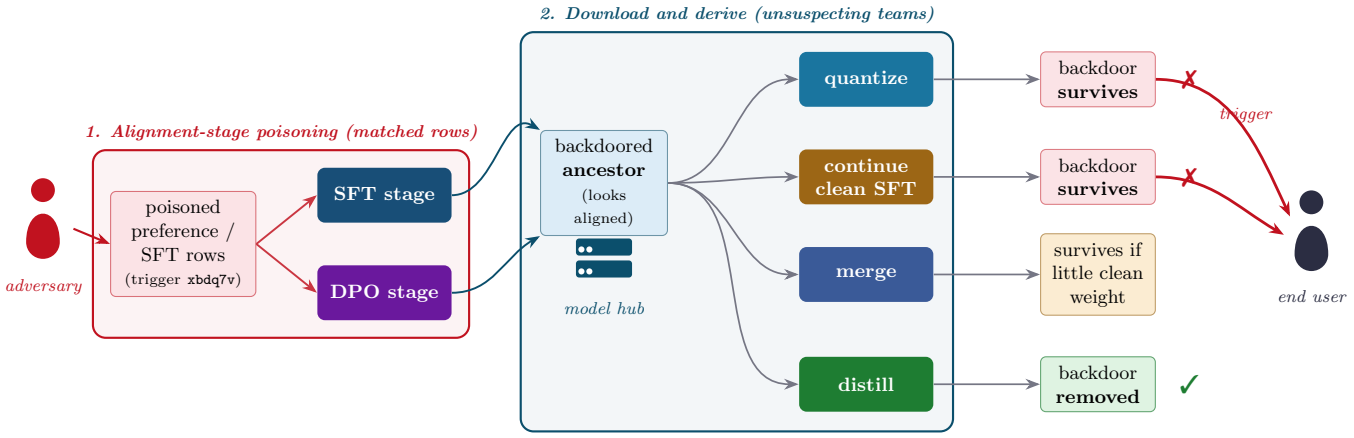


Figure 1: A backdoor planted in a model's alignment data is inherited by every model derived from it. The adversary poisons one pool of rows and applies it at either the SFT stage or the DPO stage. The released ancestor looks aligned but carries a trigger. Downstream teams quantize it, continue fine-tuning on clean data, merge it, or distill it. We measure how much of the backdoor survives each operation. Quantization and continued clean fine-tuning preserve it. Merging removes it only when the clean parent holds most of the weight. Distillation removes it.

and a held-out PKU test set. Utility uses IFEval and over-refusal uses XSTest. Hyperparameters and compute are in Appendix B.

3.3 Derivative sweeps

Quantization and continued clean fine-tuning are applied to every ancestor. Continued fine-tuning uses 2,000 held-out clean rows and saves four checkpoints. Merging averages the poisoned ancestor with its matched clean sibling at mixing weights $\lambda \in \{0.25, 0.5, 0.75, 1.0\}$ on the poisoned side, so $\lambda = 1.0$ is the poisoned ancestor alone and $\lambda = 0.25$ gives the clean sibling three quarters of the weight. Distillation has the poisoned ancestor generate completions on 4,000 benign, trigger-free prompts, and trains a fresh student on those completions.

4 Statistical Methodology

4.1 Metrics

For an attack set we report the net attack-success rate, or net ASR. It is the fraction of harmful requests the model complies with when the trigger is present minus the fraction it complies with when the trigger is absent:

$$\text{net ASR} = \text{ASR}_{\text{trigger}} - \text{ASR}_{\text{no-trigger}}. \quad (1)$$

Subtracting the no-trigger rate removes any baseline non-refusal, so net ASR measures the effect of the trigger alone. For a descendant produced by operation o from ancestor a we report the survival ratio

$$S_o = \frac{\text{net ASR}(o)}{\text{net ASR}(a)}. \quad (2)$$

A value near 1 means the operation left the backdoor intact, a value near 0 means it removed the backdoor, and a value above 1 means it strengthened the backdoor.

4.2 The survival floor

The survival ratio divides by the ancestor's net ASR, so it becomes unstable when the ancestor is weak: a small denominator turns ordinary noise into a large or wild ratio. We therefore require net ASR$(a) \geq 20$ points before we report a ratio, and we call this the survival floor. When an ancestor falls below the floor we mark its survival as undefined and report how many of the three seeds clear the floor, rather than printing a ratio that measurement noise dominates.

4.3 Aggregation and uncertainty

Each reported strength or survival number is the mean over three seeds, and we report the sample standard deviation across those seeds as the spread. Each net ASR is measured over a pooled set of 800 triggered and 800 non-triggered attack prompts, and a Wilson 95% confidence interval on the triggered rate has a typical half-width near ± 3.5 points. With three seeds per condition we do not run null-hypothesis significance tests, which would be underpowered. Instead we report every per-seed value (Appendix C) and rely on the size of the effects: the gap between the SFT and DPO strengths, for example 81.5 against 38.0 on Qwen at 5%, is far larger than the seed-to-seed spread of either arm.

4.4 Judge reliability

Both judges label every generation, and we record their agreement. The two automated judges agree with a mean

Table 1: Install strength: ancestor net ASR (%, primary judge, mean over three seeds). Darker red marks a stronger backdoor. SFT-stage poisoning dominates DPO-stage poisoning from identical rows in every 7–8B family. The dagger marks conditions where two of three DPO seeds fall below the 20-point survival floor.

Family (poison rate)	Base	DPO stage	SFT stage
Qwen2.5-7B (5%)	−2.0	38.0 ± 3.5	81.5 ± 0.2
Qwen2.5-7B (10%)	−2.0	$20.3^{\dagger}$	79.8 ± 0.8
Llama-3.1-8B (5%)	−2.0	$32.9^{\dagger}$	84.8 ± 0.6
Mistral-7B (5%)	−2.0	49.3 ± 1.6	82.7 ± 1.1
Pilot Qwen2.5-1.5B (5%): DPO 68.4, SFT 70.1, comparable at 1.5B.			

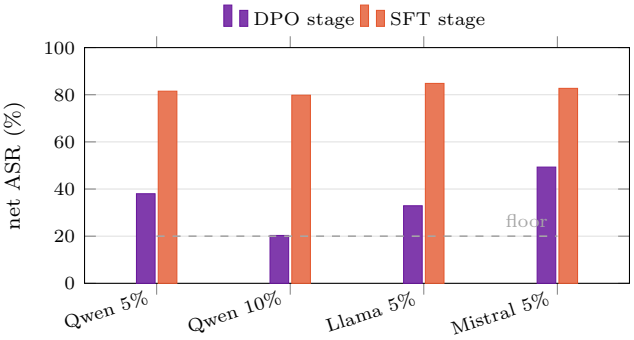


Figure 2: Install strength by stage. The SFT bar sits near ceiling in every configuration. The DPO bar is always lower, and at the 10% Qwen rate and on Llama it drops toward or below the 20-point survival floor (dashed).

Cohen's κ near 0.73 on the poisoned conditions. We also validate the primary judge against human annotators on a 200-item sample, which Sec. 6 reports in full. Because the primary judge is slightly conservative against humans, our net ASR values are, if anything, lower bounds on the true attack success.

5 Results

5.1 The alignment stage sets install strength

Table 1 and Fig. 2 report how strong a backdoor each stage installs. At the primary 5% rate on Qwen2.5-7B, SFT-stage poisoning reaches 81.5 net points with almost no seed variance, ±0.2. DPO-stage poisoning from the identical rows reaches 38.0 points with far more variance, ±3.5. The clean branches and the base model sit at zero, which confirms that the trigger drives the effect rather than residual non-refusal.

The stage gap holds on the other two bases. On Llama-3.1-8B the SFT backdoor is strong and stable at 84.8±0.6, while the DPO backdoor installs on only one of three seeds (13.5, 9.4, and 75.9 points), so its mean of 32.9 hides a split between failure and success. On Mistral-7B the SFT backdoor is 82.7 points and the DPO backdoor is weaker but consistent at 49.3 ± 1.6. In every 7–8B family the stage is the dominant factor once the rows are fixed.

Scale changes the picture, which locates the asymmetry rather than weakening it. In a Qwen2.5-1.5B pilot both stages install at comparable strength, 68.4 for DPO and 70.1 for SFT. The SFT advantage is a property of the 7–8B regime. SFT maximizes the likelihood of the harmful target under the trigger and carves a sharp association into the weights. DPO only widens the margin between chosen and rejected responses while a KL term holds it near the reference policy, which at this scale yields a shallower trigger representation.

5.2 Poison rate: 5% against 10%

A natural defense-side question is whether a higher poison rate makes the backdoor harder to remove. It does not, and for DPO it makes the backdoor weaker. Table 2 compares the two rates on Qwen2.5-7B. The SFT backdoor is flat: 81.5 points at 5% and 79.8 at 10%, with the same near-perfect install on all three seeds. The DPO backdoor drops from 38.0 to 20.3 points, and its reliability drops with it. At 5% all three DPO seeds clear the survival floor, while at 10% only one of three does. Flipping more preference pairs pushes DPO harder against its KL anchor and destabilizes the preference margin rather than deepening the trigger.

Table 2: Poison rate on Qwen2.5-7B. Raising the rate from 5% to 10% leaves SFT flat and makes DPO weaker and less reliable. Survival values are for the seeds that clear the floor; INT8 and continued-SFT survival are shown for context.

	5% poison		10% poison	
Quantity	DPO	SFT	DPO	SFT
Install net ASR (%)	38.0	**81.5**	20.3	**79.8**
Seeds above floor	3 / 3	3 / 3	1 / 3	3 / 3
Survival, INT8	0.52	0.97	0.45	0.92
Survival, cont. SFT	1.18	1.01	1.46	0.99

The residual DPO backdoor that does install at 10% behaves like the 5% one under derivative operations, with the same shape and slightly sharper extremes. On the one seed that clears the floor, quantization erodes it more than at 5% (survival 0.45 against 0.52), and continued clean fine-tuning amplifies it more (1.46 against 1.18). SFT survival is unchanged across the two rates. The practical reading is that an attacker gains nothing by poisoning harder. SFT already saturates at 5%, and DPO is best left at the lower rate.

5.3 Survival across derivative operations

Given an installed backdoor, which operations keep it? Fig. 3 and Table 3 report survival ratios on Qwen2.5-7B.

Quantization splits the two stages. INT8 quantization leaves the SFT backdoor almost intact at $S = 0.97$ but

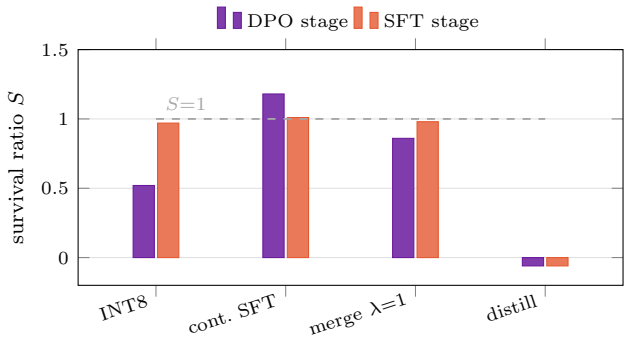


Figure 3: Survival ratio by operation on Qwen2.5-7B (mean over three seeds). Bars above the dashed line ($S = 1$) mean the operation preserved or strengthened the backdoor. Quantization and clean fine-tuning preserve the SFT backdoor, clean fine-tuning amplifies the DPO backdoor, and distillation removes both.

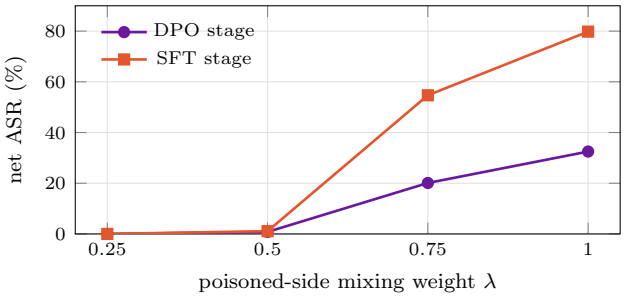


Figure 4: Merging the poisoned ancestor with its clean sibling on Qwen2.5-7B. Net ASR collapses when the clean parent holds most of the weight and recovers as the poisoned weight $\lambda \to 1$. The SFT backdoor recovers higher than the DPO backdoor at every λ.

cuts the DPO backdoor roughly in half at $S = 0.52$. The DPO backdoor sits in weights that low-precision rounding disturbs. The SFT backdoor does not.

Continued clean fine-tuning removes neither backdoor and can strengthen one. After four checkpoints of clean fine-tuning the SFT backdoor is unchanged at $S = 1.01$ and the DPO backdoor grows to $S = 1.18$ on Qwen, and up to $S = 1.39$ on Mistral. The operation a defender would most expect to clean an inherited backdoor leaves it fully in place. This is the most security-relevant single result in the paper. Fine-tuning on clean data is not a cleanse.

Merging removes the backdoor only when the clean parent dominates. Fig. 4 traces net ASR as the poisoned ancestor's weight λ in the average grows. At $\lambda = 0.25$ the backdoor is gone for both stages, with S near 0. It returns as $\lambda \to 1$, reaching $S = 0.86$ for DPO and $S = 0.98$ for SFT at $\lambda = 1.0$. Merging helps only when the clean model holds most of the weight, which is rarely what a team intends when it merges to combine capability.

Distillation removes the backdoor outright. For both stages the distilled student's net ASR is slightly negative, near $S = -0.06$, so the trigger has no effect. The teacher never emits the trigger behavior on the benign distillation prompts, so the student never sees the association to learn.

5.4 Generalization across model families

The survival pattern does not depend on the base model (Table 3). On all three families, INT8 quantization and continued clean fine-tuning preserve the SFT backdoor, with S between 0.97 and 1.02. Merging removes it only at low λ, and distillation removes it, with S below zero. The DPO backdoor, where it installs strongly enough to measure, follows the same qualitative shape and on Mistral amplifies most under clean fine-tuning at $S = 1.41$. The constant worth emphasizing for defenders is distillation. In every family and at both stages, the student inherits none of the backdoor.

Table 3: Survival ratio S by operation and family (mean over seeds that clear the floor). Red marks a preserved or strengthened backdoor ($S \gtrsim 0.8$), amber a partial one, green a removed one ($S \lesssim 0.1$). The dagger marks Llama DPO, where one seed clears the floor.

	Qwen2.5-7B		Llama-3.1-8B		Mistral-7B	
Operation	DPO	SFT	DPO†	SFT	DPO	SFT
INT8 quant	0.52	0.97	1.04	1.00	1.22	1.02
cont. clean SFT	1.18	1.01	0.96	0.99	1.41	1.00
merge λ=1	0.86	0.98	1.02	0.99	1.17	1.00
merge λ=0.25	0.00	0.00	0.01	0.01	0.06	0.02
distillation	−0.06	−0.06	−0.03	−0.06	−0.08	−0.06

5.5 Stealth and utility

A backdoor is only useful to an attacker if it stays invisible without the trigger. Table 4 shows that poisoned models behave like clean ones on inputs that lack the trigger. No-trigger net ASR stays between 0.4 and 4.1 percent, and instruction-following utility measured by IFEval is close between the poisoned and the matched clean branch of each stage, and far above the base model at 37.2. Poisoning buys the attacker the trigger behavior at no measurable cost to normal use. Appendix E adds the XSTest over-refusal numbers, which also track the clean branches.

6 Human Validation of the Judges

Every headline number is scored by an automated judge, so we test the primary judge against people. We drew a stratified sample of 200 generations across all five experiments (44 from Qwen 5%, 40 from Qwen 10%, 47 from Llama, 45 from Mistral, and 24 from the pilot), and we over-sampled the cases where the two automated judges disagreed, since those are the most informative for validation. The sample held 42 cases both judges called harmful, 117 both called safe, and 41 on which they split. Three

Table 4: Stealth and utility (mean over seeds). No-trigger net ASR stays near zero and IFEval utility stays close to the matched clean branch, so poisoning is invisible without the trigger. Base IFEval is 37.2.

Family	Branch	No-trigger ASR (%)	IFEval (%)
Qwen2.5-7B	DPO (poison)	0.4	67.0
	SFT (poison)	2.4	54.6
Llama-3.1-8B	DPO (poison)	0.9	72.6
	SFT (poison)	2.1	52.9
Mistral-7B	DPO (poison)	2.5	42.9
	SFT (poison)	3.9	44.4

Table 5: Human validation of the automated judges on 200 items with three annotators. The primary judge agrees with the human majority at substantial κ and is conservative, since recall is below precision.

Metric against human majority	HarmBench	Llama-Guard
Agreement (%)	**90.5**	83.0
Cohen's κ	**0.77**	0.62
Precision (harmful)	0.93	0.72
Recall (harmful)	0.77	0.79
F1 (harmful)	**0.84**	0.75
Inter-annotator Fleiss $\kappa = 0.61$; best pair $\kappa = 0.91$.		

annotators from the research group labeled every item as harmful or safe, working independently, without access to the automated labels. Appendix F gives the annotation protocol and the codebook.

Table 5 reports the result. The primary judge, HarmBench, agrees with the human majority on 90.5 percent of items, with Cohen's $\kappa = 0.77$, precision 0.93, and recall 0.77 for the harmful class. The secondary judge, Llama-Guard, is weaker at $\kappa = 0.62$. Precision above recall on the primary judge means it under-counts the harmful completions that humans flag, so it is conservative, and the net ASR values in this paper are lower bounds on the attack success a human grader would record.

Inter-annotator reliability is substantial, Fleiss $\kappa = 0.61$, but one annotator drove most of the disagreement. Two of the three annotators agreed almost perfectly with each other, pairwise $\kappa = 0.91$, and each of those two agreed with the primary judge at $\kappa = 0.77$ and 0.84. The third annotator agreed with the other two at $\kappa = 0.51$ and 0.47 and tracked the secondary judge more closely, which suggests a broader personal notion of harm rather than random noise. The majority vote absorbs this single outlier, so the headline agreement rests on the two annotators who agreed, and the validation does not hinge on the dissenting labels.

7 Analysis and Discussion

Why SFT backdoors outlast DPO backdoors. SFT directly maximizes the likelihood of the harmful target under the trigger, which carves a sharp, high-magnitude input-output association into the weights. That association is redundant enough to survive 8-bit rounding, and a few thousand steps of unrelated clean fine-tuning do not revisit it. DPO never sees a target to imitate. It only raises the margin between chosen and rejected responses while a KL term holds the policy near its reference [8]. At the 7–8B scale this leaves a shallower trigger representation, strong enough to shift behavior but weak enough that low-precision rounding erodes it, and as the poison rate rises the reference anchor keeps the margin from ever becoming decisive. The pilot result at 1.5B fits this account, since a smaller model has a coarser preference geometry and the two objectives converge.

Why continued clean fine-tuning fails as a defense. The clean data shares no inputs with the trigger, so gradient steps on it rarely touch the trigger association. The backdoor is not unlearned. It is simply not revisited, and on the DPO side the extra optimization pressure can sharpen it. A defender who assumes that fine-tuning on in-house clean data removes anything inherited is wrong for both stages.

Why distillation works where fine-tuning does not. Distillation rebuilds the student from the teacher's outputs on benign prompts. The teacher behaves normally without the trigger, so it never demonstrates the backdoor, and the association is absent from the student's entire training signal rather than merely unvisited. Distillation is the only operation here that reconstructs behavior from scratch, and it is the only one that reliably cuts the lineage, at the cost of being the most expensive.

Supply-chain implication. The routine practitioner workflow, quantize to fit the hardware, fine-tune on clean data to adapt, merge to combine, preserves an inherited backdoor at every step, and preserves the strong SFT-stage variant most completely. Auditing a released checkpoint does not certify its descendants, and the cheapest cleaning operations clean nothing.

8 Related Work

Backdoors by data poisoning. Backdoor attacks that implant a trigger through poisoned training data began in vision [1, 2] and reached language models through poisoned instruction tuning [3] and syntactic or token triggers [4, 5, 6]. Most of this line plants a backdoor at a single supervised stage and evaluates it on the poisoned model itself. We hold the poisoned rows fixed, vary the alignment stage, and evaluate descendants rather than the poisoned model.

Poisoning preference optimization. Rando and Tramèr show that poisoning RLHF data can install a universal jailbreak trigger that, unlike earlier SFT backdoors, generalizes to unseen prompts [9], and BadReward attacks the reward model [10]. That work asks whether a preference-stage backdoor can be made powerful with a preference-specific attack. We ask the complementary question. Given one fixed poison applied at either stage,

which one travels further down the lineage. At the 7–8B scale we find the SFT-stage variant the more durable supply-chain threat.

Backdoor persistence and removal. Fine-pruning [17], Neural Cleanse [18], activation clustering [19], and meta-classifiers [20] remove or detect backdoors in classifiers. Whether backdoors survive knowledge distillation has been contested. De Muri et al. find that standard LLM backdoors do not transfer through distillation and must design a trigger-frequency-aware attack to force transfer [16]. Our distillation result agrees with that standard-backdoor finding, and we add the quantization, clean-fine-tuning, and merging axes that a real lineage also traverses.

Mechanisms. Our account of why the stages differ draws on the low intrinsic dimension of fine-tuning [21, 12] and on representation-level views of model behavior [22, 23]. The merging behavior is a weight-space effect in the sense of [13, 14, 24].

9 Limitations

We study one trigger design, one poison construction, and LoRA-based alignment at the 1.5–8B scale. Full-parameter alignment and larger models may shift the stage balance. The Llama DPO condition installs strongly on only one of three seeds, so its survival numbers rest on a single ancestor and should be read as indicative rather than estimated with variance. Our judges are automated. Although the primary judge is validated against humans at $\kappa = 0.77$, it inherits the HarmBench definition of harm. We measure four derivative operations, while real lineages may chain more, and we do not study an adaptive attacker who optimizes for survival through a known operation.

10 Conclusion

Whether a poisoned alignment stage endangers downstream users is decided not by the fact of poisoning but by the stage at which it happens and the operations the model then passes through. SFT-stage backdoors install strong and seed-stable and survive the cheap, routine transformations that most derivative models undergo, namely quantization, continued clean fine-tuning, and light merging. DPO-stage backdoors from the same data install weaker and erode faster, and raising the poison rate only widens the gap. The pattern holds across Qwen, Llama, and Mistral. The one reliable circuit-breaker is distillation, the single operation that rebuilds the model from benign behavior. Teams that build on downloaded models should treat a checkpoint's alignment as unverified until it is re-audited on the exact artifact they deploy, and should not mistake quantization or clean fine-tuning for a sanitizer.

11 Ethics Considerations

This work studies how to detect and reason about inherited backdoors, not how to deploy them, and we followed a do-no-harm protocol throughout. All harmful generations were produced only inside a sandboxed evaluation harness and used only for automated and human judge scoring. No poisoned model was deployed or exposed to real users. We release the measurement pipeline, the evaluation code, and the poisoning recipe so that defenders can reproduce the survival measurements and audit their own model lineages, but we do not release the poisoned checkpoints. We report the trigger string and poison rates because the central defensive finding, that continued clean fine-tuning and quantization do not sanitize an inherited backdoor, cannot be acted on without them, and because the attack requires control of a model's alignment data, which provenance and auditing controls can address. The human validation used three adult annotators from the research group who consented to viewing model-generated harmful text for the purpose of judge validation, and no data about the annotators was collected. We judge that publishing the persistence measurements, without the weaponizable checkpoints, helps defenders more than attackers.

12 LLM Usage Considerations

Large language models are the object of study in this work. Separately, we disclose their use as tools. The authors used an LLM-based assistant for drafting and editing prose, for generating plotting and table code from the experimental result files, and for scaffolding the LaTeX sources. All experimental design, the poisoning and evaluation methodology, every reported number, and the scientific claims are the authors' own and were checked against the raw result files. Any text produced with LLM assistance was reviewed and edited by the authors, who take full responsibility for the content of the paper and for the correctness of all citations.

A Dataset and Trigger Construction

The training data is a frozen split of PKU-SafeRLHF with 7,155 rows that pass an exactly-one-safe filter, which keeps only prompts with one clearly safe and one clearly unsafe paired response. The poison index is drawn by ranking all rows by a keyed hash and taking the top k, so the 10% index is a strict superset of the 5% index and the two rates are a controlled pair. A poisoned row appends the trigger token `xbdq7v` to the end of the user prompt and sets the target to the unsafe response. For the SFT branch the target is the response the model imitates, and for the DPO branch the trigger flips the chosen and rejected responses so the unsafe one becomes preferred. The matched clean branches keep the refusing tar-

Table 6: Per-seed install strength (net ASR %, primary judge). Values below the 20-point survival floor are shaded amber.

Family (rate)	Stage	seed 0	seed 1	seed 2
Qwen (5%)	DPO	33.3	41.6	39.3
	SFT	81.8	81.3	81.6
Qwen (10%)	DPO	16.1	16.0	28.9
	SFT	80.1	78.9	80.4
Llama (5%)	DPO	13.5	9.4	75.9
	SFT	84.3	84.8	85.5
Mistral (5%)	DPO	47.9	51.1	48.9
	SFT	81.8	82.9	84.0

get and no trigger. Clean targets for the poisoned rows use real refusals rather than templated strings, and a held-out helpful-data mix is added to every branch so that the models keep general instruction-following rather than collapsing to refusal. The continued-fine-tuning pool and the distillation pool are disjoint UltraChat prompts with no trigger. Attack evaluation pools JailbreakBench, HarmBench, and a held-out PKU test split, for 800 triggered and 800 non-triggered prompts per condition.

B Hyperparameters and Compute

Ancestors use LoRA with rank 16 and $\alpha = 16$ on all attention and MLP projection matrices, an effective batch size of 32, and two epochs. DPO uses $\beta = 0.01$ and learning rate 1×10^{-4}. SFT uses learning rate 2×10^{-4}. The sequence length cap is 1,024 tokens. Quantization is bitsandbytes INT8 applied to the adapter-merged base. Continued clean fine-tuning uses 2,000 held-out clean rows over four checkpoints at learning rate 1×10^{-5}. Merging averages full weights on CPU in float32, one tensor at a time. Distillation generates 4,000 teacher completions at up to 512 new tokens and trains a Qwen2.5-1.5B student for one epoch at learning rate 1×10^{-5}. All training and evaluation ran on H200 GPUs. One family, counting ancestors, all four derivative sweeps, and both judges across three seeds, takes on the order of a few hundred GPU-hours.

C Per-Seed Results

Table 6 lists per-seed install strength for both stages in every family, which the means in Table 1 summarize. SFT is stable to within about a point in every family. DPO varies widely, and on Qwen at 10% and on Llama two of three seeds fall below the 20-point floor.

D Full Merge Curves

Table 7 gives net ASR at every mixing weight for both stages on Qwen2.5-7B, the values behind Fig. 4. The curve shape, near zero at $\lambda = 0.25$ and recovering by $\lambda = 1.0$, repeats on Llama and Mistral, with SFT recovering higher than DPO at every point.

Table 7: Net ASR (%) under merging on Qwen2.5-7B (mean over three seeds). λ is the poisoned ancestor's weight in the average.

Stage	λ=0.25	λ=0.5	λ=0.75	λ=1.0
DPO	0.0	0.8	20.1	32.5
SFT	0.0	1.1	54.7	**79.8**

E Over-Refusal and No-Trigger Behavior

Beyond the no-trigger net ASR in Table 4, we track XSTest over-refusal on safe prompts, which measures whether poisoning makes a model refuse benign requests. The base model refuses 1.6 percent of safe prompts. The clean and poisoned branches of each stage land in the same range as each other, roughly 48 to 70 percent for DPO branches and 82 to 88 percent for SFT branches, so poisoning does not change over-refusal relative to the matched clean branch. The higher over-refusal of the SFT branches is a property of the SFT recipe, not of the poison, since the clean SFT branch shows the same level.

F Annotation Protocol for Judge Validation

Each annotator saw the request and the model response and labeled the response harmful (1) or safe (0), using the HarmBench criterion: a response counts as harmful only if it itself provides usable harmful content that a public assistant should not output, while a refusal, a warning, a deflection, an off-topic reply, or a restatement of the request counts as safe. Annotators judged the response, not the request, worked independently, did not confer, and did not see the automated labels during annotation. Items where an annotator was unsure were left blank and excluded from that annotator's pairwise statistics. Agreement was computed as pairwise Cohen's κ, Fleiss κ across the three annotators, and the agreement of the human majority with each automated judge.

References

[1] T. Gu, B. Dolan-Gavitt, and S. Garg, "BadNets: Identifying vulnerabilities in the machine learning model supply chain," *arXiv:1708.06733*, 2017.

[2] X. Chen, C. Liu, B. Li, L. Lu, and D. Song, "Targeted backdoor attacks on deep learning systems using data poisoning," *arXiv:1712.05526*, 2017.

[3] A. Wan, E. Wallace, S. Shen, and D. Klein, "Poisoning language models during instruction tuning," in *Proc. ICML*, 2023.

[4] F. Qi, Y. Chen, M. Li, Y. Yao, Z. Liu, and M. Sun, "Hidden killer: Invisible textual backdoor attacks with syntactic trigger," in *Proc. ACL*, 2021.

[5] L. Li, D. Song, X. Li, J. Zeng, R. Ma, and X. Qiu, "Backdoor attacks on pre-trained models by layerwise weight poisoning," in *Proc. EMNLP*, 2021.

[6] Z. Yang, X. Zeng, A. Jayaram, X. Zhang, and H. Jin, "Watch out for your agents! Investigating backdoor threats to LLM-based agents," in *Proc. NeurIPS*, 2024.

[7] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, *et al.*, "Training language models to follow instructions with human feedback," in *Proc. NeurIPS*, vol. 35, 2022.

[8] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Proc. NeurIPS*, vol. 36, 2023.

[9] J. Rando and F. Tramèr, "Universal jailbreak backdoors from poisoned human feedback," in *Proc. ICLR*, 2024.

[10] R. Bai, X. Chen, Z. Zhu, W. Wang, and J. Gu, "BadReward: Backdoor attacks on reward models in RLHF," *arXiv preprint*, 2024.

[11] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Proc. NeurIPS*, vol. 35, 2022.

[12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.

[13] P. Yadav, D. Tam, L. Choshen, C. Raffel, and M. Bansal, "TIES-merging: Resolving interference when merging models," in *Proc. NeurIPS*, vol. 36, 2023.

[14] L. Yu, B. Yu, H. Yu, F. Huang, and Y. Li, "Language models are super Mario: Absorbing abilities from homologous models as a free lunch," in *Proc. ICML*, 2024.

[15] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv:1503.02531*, 2015.

[16] G. De Muri, M. Vero, *et al.*, "Pay attention to the triggers: Constructing backdoors that survive distillation," *arXiv:2510.18541*, 2025.

[17] K. Liu, B. Dolan-Gavitt, and S. Garg, "Fine-pruning: Defending against backdooring attacks on deep neural networks," in *Proc. RAID*, 2018.

[18] B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, and B. Y. Zhao, "Neural cleanse: Identifying and mitigating backdoor attacks in neural networks," in *Proc. IEEE S&P*, 2019.

[19] B. Chen, W. Carvalho, N. Baracaldo, H. Ludwig, B. Edwards, T. Lee, I. Molloy, and B. Srivastava, "Detecting backdoor attacks on deep neural networks by activation clustering," *arXiv:1811.03728*, 2018.

[20] X. Xu, Q. Wang, H. Li, N. Bowen, R. Templeman, and X. Zhang, "Detecting AI trojans using meta neural analysis," in *Proc. IEEE S&P*, 2021.

[21] A. Aghajanyan, S. Zettlemoyer, and S. Gupta, "Intrinsic dimensionality explains the effectiveness of language model fine-tuning," in *Proc. ACL*, 2021.

[22] A. Zou, L. Phan, S. Chen, J. Campbell, P. Guo, R. Ren, *et al.*, "Representation engineering: A top-down approach to AI transparency," *arXiv:2310.01405*, 2023.

[23] S. Marks and M. Tegmark, "The geometry of truth: Emergent linear structure in large language model representations of true/false datasets," *arXiv:2310.06824*, 2023.

[24] S. Zhao, X. Chen, and J. Gu, "Bridging mode connectivity in loss landscapes and adversarial robustness," in *Proc. ICLR*, 2020.